

Q1. Adjacency Matrix – Representation and Inference

Consider the following adjacency matrix of a directed graph G with vertex set $V = \{1, 2, 3, 4, 5\}$:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- Write down the set of directed edges E represented by the matrix A .
- Draw the directed graph G corresponding to the matrix A .
- For each vertex $i \in \{1, 2, 3, 4, 5\}$, compute:
 - the out-degree of i (number of 1's in row i),
 - the in-degree of i (number of 1's in column i).
- Based only on the adjacency matrix (without redrawing the graph), answer:
 - Is there any vertex with no incoming edges? If yes, identify it.
 - Is there any vertex with no outgoing edges? If yes, identify it.
 - Can the graph contain a self-loop at any vertex? Justify from the matrix.
- Using the structure of A only (you may reason row-wise/column-wise), comment whether:
 - The graph can be strongly connected.
 - The graph must contain at least one path of length ≥ 2 .

Give a brief justification for each answer.

Q2. Adjacency List – Representation and Inference

A graph G with vertex set $V = \{A, B, C, D, E, F\}$ is represented using the following adjacency list (assume it is an undirected graph):

A : B, C
 B : A, D, E
 C : A, F
 D : B, E
 E : B, D, F
 F : C, E

- Rewrite the adjacency list in a more formal mathematical form by listing the set of vertices V and the set of edges E .
- Using only the adjacency list:
 - Compute the degree of each vertex.
 - Verify that the Handshaking Lemma

$$\sum_{v \in V} \deg(v) = 2|E|$$

holds for this graph.

- Without drawing the graph, determine whether G is connected. Clearly explain how you use the adjacency list to justify your answer (for example, by performing a mental BFS/DFS using the list).
- Suppose we delete the edge (B, E) from the graph (and update the adjacency list accordingly).
 - Will the graph still remain connected? Argue based purely on the updated adjacency list.
 - Identify a vertex (if any) whose degree reduces from 3 to 2 and comment on how this changes the number of possible simple paths starting from that vertex.

- (e) Based on the original adjacency list (before deletion), reason whether the graph can contain:
- At least one cycle of length 3.
 - At least one cycle of length 4.

You may trace possible sequences of vertices directly from the adjacency lists without drawing the graph.

Q3. Breadth-First Search (BFS) – Adjacency Matrix and Step-by-Step Trace

Consider the following directed graph G with vertex set $V = \{1, 2, 3, 4, 5, 6\}$, represented using its adjacency matrix A :

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

An entry $A[i][j] = 1$ indicates a directed edge from vertex i to vertex j . Assume vertices are explored in increasing numerical order whenever there is a choice.

- Starting from source vertex 1, perform a **BFS traversal**. Write the BFS order of visiting vertices.
- Construct a **step-by-step BFS trace table** with the following columns:

Step No.	Current Vertex	Queue Contents	Newly Discovered Vertices
----------	----------------	----------------	---------------------------

Fill this table for the entire traversal starting from vertex 1.

- Using only the adjacency matrix and your BFS trace:
 - Determine the **level (distance in edges)** of each vertex from source 1.
 - Identify any vertex that is not reachable from vertex 1 (if any).
- Construct the **BFS tree** rooted at vertex 1 based on your traversal:
 - List the parent of each vertex in the BFS tree.
 - Clearly separate **tree edges** from **non-tree edges** using the matrix A .

Q4. Depth-First Search (DFS) – Adjacency List, Recursion Calls and Backtracking

Consider the following directed graph G with vertex set $V = \{A, B, C, D, E, F\}$, represented using an adjacency list. Assume that when exploring neighbors, they are visited in **alphabetical order**.

A : B, C
 B : D, E
 C : F
 D : (none)
 E : F
 F : (none)

A recursive DFS is performed using the function:

```
DFS(u):
    mark u as visited
    for each v in Adj[u] in alphabetical order do
        if v is not visited then
            DFS(v)
```

The initial call is DFS(A).

- Write the **DFS visiting order** of the vertices (preorder), starting with the call DFS(A).

- (b) Construct a detailed **recursive call trace table** with the following columns:

Step No.	Current Call (DFS(u))	Action (Explore / Return)	Call Stack Contents
----------	-----------------------	---------------------------	---------------------

Capture every time a new DFS call is made and every time a call returns (backtracks).

- (c) Based on your trace, answer:

- Total number of recursive calls** made (including the initial call).
- Total number of backtracking events** (i.e., the number of times a DFS call finishes and control returns to its caller).

- (d) Draw the **DFS recursion call tree**, where:

- Each node in the recursion tree represents a call $\text{DFS}(X)$.
- A directed edge from $\text{DFS}(U)$ to $\text{DFS}(V)$ represents that $\text{DFS}(V)$ was called from $\text{DFS}(U)$.

Label the nodes in the order they are first called.

- (e) For each vertex, write down its:

- Discovery time (time when the vertex is first visited),
- Finish time (time when the DFS call for that vertex returns for the last time).

(You may assume a time counter that increments by 1 on each discovery and each return.)

Q5. Matrix Exponentiation – Counting Paths in a Graph

A directed graph G with $V = \{1, 2, 3, 4\}$ is represented by the adjacency matrix:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- Compute A^2 and A^3 .
- Interpret the values of $A^2[1][3]$ and $A^3[1][4]$ in terms of number of walks between the corresponding vertices.
- Using your results, determine the number of distinct walks of exactly length 5 from vertex 1 to vertex 1, without manually enumerating paths. (Hint: Use cyclic properties of powers of A .)

Q6. Modified DFS – Path Searching Under Constraints

A social-network graph models an information flow among users:

$$A \rightarrow B, C; \quad B \rightarrow D; \quad C \rightarrow D, E; \quad D \rightarrow F; \quad E \rightarrow F$$

A rumor must be spread from user A to user F , but the spread is **not allowed through any user whose out-degree is greater than or equal to 2**.

- Modify DFS so that such users are skipped (but still marked visited so they are not revisited).
- Perform the modified DFS starting from A and show the full stepwise trace (recursive calls and returns).
- State whether spreading to F is possible under the given restriction. If yes, list one valid path; if no, justify why.

Q7. Warshall's Algorithm – Detecting Indirect Dependencies

A software build system has the following dependency graph:

$$1 \rightarrow 2; \quad 2 \rightarrow 3; \quad 3 \rightarrow 4; \quad 4 \rightarrow 2$$

- Construct the initial reachability matrix $R(0)$.

- (b) Apply **Warshall's Algorithm** step by step to compute $R(1)$, $R(2)$, $R(3)$, $R(4)$, and the final transitive closure.
- (c) Based only on the final matrix, list:
- All pairs of modules involved in dependency cycles.
 - All strongly connected components.
 - Whether it is safe to perform a topological sorting on this dependency graph (justify).

Q8. Transitive Computation – Stabilizing Reachability Using Recurrence

Let $T(n)$ denote the reachability matrix of a directed graph G after applying Warshall's recurrence for n iterations:

$$T(n)[i][j] = T(n-1)[i][j] \vee (T(n-1)[i][k] \wedge T(n-1)[k][j]),$$

where $k = n$.

For a graph with $V = \{1, 2, 3, 4, 5\}$, assume the following matrices were obtained:

$$T(3) = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad T(4) = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T(5) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

- Determine whether **the transitive closure has been reached** by comparing $T(4)$ and $T(5)$.
- State the precise condition under which computation can be stopped early instead of continuing to $T(|V|)$.
- List the vertices that become newly reachable between $T(3)$ and $T(5)$ and describe their real-world meaning in terms of propagation/indirect access.